

AI Use Declaration Form

Course: Introduction to Artificial Intelligence

Lab Title: Lab 2 Supervised and unsupervised learning

Student Name: Fatma Ali Abdourahmane

Student ID: 30592027

GitHub Repository Link: _____

Date Submitted: 03/07/2026

1. AI Use Summary

Question	Student Response
Did you use any AI tool for this lab?	YES
Estimated percentage of the work influenced by AI	%15
Did you attach evidence of AI use where applicable?	YES

2. Details of AI Use

Complete the table below for each AI tool used. Add more rows where necessary.

Name of AI Tool Used	Purpose for Using the Tool	Prompt or Instruction Given to the Tool	Part(s) of the Work Influenced by the Tool	How I Verified, Edited, or Improved the AI Output
Copilot	To help	Explain the lab instructions	Visualizations	Fixed on my own oreference of design

3. Attachment of AI Output Evidence

Where required, attach evidence of AI use. Tick all that apply. Add more rows where necessary.

Evidence Type	Attached? (Yes, No, N/A)	File Name
AI-generated draft		
Prompt history		
Screenshot of AI interaction	YES	Screenshot
Exported AI conversation		
AI-generated code snippet		
Revised version showing student input		

4. Student Declaration

I declare that:

Declaration Statement	Response (Yes/No)
The submitted work is my own work.	YES
Any use of AI tools has been clearly declared in this form.	YES
The prompts, instructions, and AI-generated outputs have been disclosed where applicable.	YES
I reviewed, tested, edited, and improved any AI-generated content before submission.	YES
My AI usage does not exceed 25% of the entire work.	YES
I understand that undeclared or excessive AI use may be treated as academic misconduct.	YES

Student Signature: Fatma Ali Abdourahmane

Date: 03/07/2027

AI output evidence and external sources:

Here's a clear breakdown of how to understand and follow **lab instructions**. Think of them as a recipe for science or data analysis — they tell you what to do, in what order, and why each step matters.

1 • Read the Objective

Understand what the lab is trying to achieve.

- Identify the **main question** or hypothesis
- Note the **target variable** (e.g., tips, obesity level)
- Clarify whether it's regression, classification, or clustering

2 • Check the Dataset

Inspect the data before doing any modeling.

- Look at the **shape** (rows × columns)
- Spot **missing values** or wrong entries
- Decide which features are **numeric** v  **categorical**

Distribution of Tips

Python ^

 Copy

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(taxi['tip'], bins=40, kde=True)
plt.title("Tip Distribution")
plt.xlabel("Tip Amount")
plt.ylabel("Frequency")
plt.show()
```

Confusion Matrix

Python ^

 Copy

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_val, y_preds)
```



Silhouette Scores

Python ^

Copy

```
from sklearn.metrics import silhouette_score

scores = []
for k in range(2, 6):
    km = KMeans(n_clusters=k, random_state=0)
    labels = km.fit_predict(X_train)
    scores.append(silhouette_score(X_train, labels))

plt.plot(range(2, 6), scores, marker='x', color='orange')
plt.title("Silhouette Scores")
plt.xlabel("k")
plt.ylabel("Score")
plt.show()
```

1. Encode categorical columns

- **Binary yes/no columns** (`family_history_with_overweight` , `FAVC` , `SMOKE` , `SCC`):
Turn "yes" into 1 and "no" into 0.

Python ^

Copy

```
binary_cols = ['family_history_with_overweight', 'FAVC', 'SMOKE', 'SCC']
for col in binary_cols:
    obesity[col] = obesity[col].map({'yes':1, 'no':0})
```

- **Ordinal columns** (`CAEC` , `CALC`):
These have a natural order (no → sometimes → frequently → always). Use **OrdinalEncoder**.

Python ^

Copy

```
from sklearn.preprocessing import OrdinalEncoder
encoder = OrdinalEncoder()
obesity[['CAEC', 'CALC']] = encoder.fit_transform(obesity[['CAEC', 'CALC']])
```

1. Encode categorical columns

- For **binary yes/no columns** like `family_history_with_overweight`, `FAVC`, `SMOKE`, `SCC`, you change “yes” to 1 and “no” to 0. This makes them numeric and easy for models to use.
- For **ordinal columns** like `CAEC` and `CALC`, the values have a natural order (no → sometimes → frequently → always). You use an **OrdinalEncoder** so the model understands the order.
- For **nominal columns** like `Gender` and `MTRANS`, there is no order. You use **One-Hot Encoding** (dummy variables) so each category becomes its own column.

2. Engineer a domain feature (BMI)

- You can create **BMI = Weight / Height²**.
- This feature is strongly related to obesity level. Including it can improve accuracy, but it may make the task “too easy” because BMI overlaps with the target. So you should discuss whether it’s fair to include depending on your goal.

Here are some YouTube tutorials that explain encoding, feature engineering (like BMI), and scaling — they’ll help you do the preprocessing part of your lab. These videos walk through binary encoding, ordinal vs nominal encoding, one-hot encoding, label encoding, and scaling with StandardScaler.

Helpful YouTube Links

- **Understanding Data Transformation: Normalization, Encoding, and Feature Scaling**
Covers handling missing values, normalization vs standardization, label encoding, one-hot encoding, and building preprocessing pipelines.
 Watch here ([youtube.com in Bing](#))
- **Data Transformation — Normalization, Encoding, Feature Scaling**
Explains different encoding types (binary, ordinal, nominal), scaling methods, and their impact on model performance.
 Watch here ([youtube.com in Bing](#))
- **Feature Encoding Techniques (GeeksforGeeks YouTube)**
Demonstrates binary mapping, label encoding for ordinal data, and one-hot encoding for nominal features with examples.



https://www.youtube.com/results?search_query=https%3A%2F%2Fwww.youtube.com%2Fwatch%3Fv%3D9yl6-HEY7_s

<https://www.youtube.com/watch?v=sxEqtjLC0aM>

<https://www.youtube.com/watch?v=pYVScuY-GPk>